

Fedimint Confidential eCash

Gazelle

July 2026

1 Introduction

Electronic cash (eCash) aims to provide a digital analogue of physical cash, enabling users to perform electronic payments while preserving strong privacy guarantees. Traditional eCash systems focus on anonymity by preventing the issuer from linking the withdrawal of a coin to its subsequent spending through the use of blind signatures. More recent constructions extend this notion of privacy by providing **confidential eCash**, where not only the identity of the user remains protected, but also the **value of the eCash note**. In such systems, the denomination is cryptographically hidden from all parties except those authorized to learn it, while still allowing the correctness of transactions to be publicly verified.

Designing a confidential blind eCash system requires satisfying several fundamental security and practical properties. First, the system should guarantee **unlinkability**, ensuring that an issued note cannot be linked to the transaction in which it is later spent, thereby preserving the user's anonymity. Second, it must satisfy **unforgeability**, meaning that every valid eCash note must originate from the mint and that no adversary can create additional spendable notes or increase their value. Third, the protocol should support **offline transactions**, allowing payments to be completed without requiring communication with the mint during the spending phase. Finally, the construction should be **practical**, with cryptographic proofs and transaction sizes that remain compact and computational costs that are sufficiently low for deployment in real-world payment systems.

This document introduces the concept of confidential blind eCash generated within a federation and discusses the cryptographic techniques that enable both transaction privacy and value confidentiality while maintaining strong security guarantees and practical efficiency.

2 Threshold Anonymous Credentials

This project builds upon a modified version of the Coconut credential scheme [1], as adapted and deployed by Nym Technologies [2]. Coconut is a threshold anonymous credential system that enables users to obtain credentials from

multiple authorities while preserving privacy through selective disclosure and unlinkability. Nym extends the original Coconut construction by introducing optimizations aimed at improving both privacy and efficiency in practical deployments. In particular, Nym redesigns the credential structure to reduce the computational and communication overhead associated with issuance and verification, making the system more suitable for large-scale, real-world anonymous networks. Furthermore, the credential format is adapted to better support unlinkable authentication across multiple sessions, preventing credential presentations from being correlated while maintaining compatibility with threshold issuance.

At a high level, a Coconut credential consists of a set of cryptographic commitments to user attributes together with a threshold signature issued jointly by a committee of authorities. The commitments allow users to prove statements about their attributes in zero knowledge without revealing the attributes themselves, while the threshold signature guarantees that the credential has been legitimately issued even if only a quorum of authorities participates. Nym retains these core cryptographic principles but modifies the credential structure and issuance protocol to better integrate with its privacy-preserving network architecture, reducing proof sizes and verification costs while preserving the security guarantees of the original Coconut scheme. The following sections describe the credential structure adopted in this project and highlight the differences with the original Coconut construction.

3 Confidential Federated eCash

This chapter presents the proposed confidential federated eCash protocol, describing the lifecycle of an eCash note from issuance to spending. The protocol combines threshold issuance, blind signatures, and confidential commitments to provide privacy, unlinkability, and protection against forgery while ensuring that the value of each note remains hidden throughout its lifetime. For clarity, the protocol is divided into four phases. The issuance request phase describes how a user prepares a confidential eCash request, blind signing explains how the federation of mints jointly issues a blind signature, unblinding details how the user obtains a valid spendable note, and spending presents the verification process used to redeem the eCash while preserving user privacy.

3.1 Requesting Issuance of eCash

A single instance of the confidential eCash based on Nym's credential will have the following structure:

$$\begin{aligned}
pc &= m_1 G_p + r_p H_p && \text{(Pedersen commitment to the amount)} \\
c_m &= m_1 H_1 + m_2 H_2 + m_3 H_3 + r_m G && \text{(commitment to all attributes)} \\
h &= H_{G_1}(c_m) && \text{(derived signature base)} \\
c_j &= m_j h + r_j G, \quad j \in \{1, 2, 3\} && \text{(blinded attributes)}
\end{aligned}$$

Such a structure will limit us in practice when providing proofs of knowledge when multiple instances are to be requested at the same time. The well-formedness of $\vec{Y} = (pc, c_m, c_1, c_2, c_3)$ is the statement we must prove when creating the issuance requests structured as below:

$$\varphi_h(m_1, m_2, m_3, r_p, r_m, r_1, r_2, r_3) = \begin{pmatrix} m_1 G_p + r_p H_p \\ m_1 H_1 + m_2 H_2 + m_3 H_3 + r_m G \\ m_1 h + r_1 G \\ m_2 h + r_2 G \\ m_3 h + r_3 G \end{pmatrix} = \begin{pmatrix} pc \\ c_m \\ c_1 \\ c_2 \\ c_3 \end{pmatrix}.$$

The user proves knowledge of a preimage using a Sigma protocol made non-interactive via Fiat-Shamir:

$$\vec{\rho} \xleftarrow{\$} K^8, \quad \varphi_h(\vec{\rho}) = \vec{R} \quad c = H_s(\vec{Y} \parallel \vec{R}), \quad \vec{s} = \vec{\rho} + c\vec{x}$$

The verifier checks:

$$\varphi_h(\vec{s}) \stackrel{?}{=} c\vec{Y} + \vec{R}.$$

A naive approach to requesting multiple eCash notes simultaneously results in a proof size that grows linearly with the number of requested notes, which quickly becomes impractical. In the proposed construction, each issuance request requires a proof of approximately 496 bytes per note. When combined with Fedimint's consensus message size limit of 64 KB, this overhead significantly restricts the number of notes that can be issued within a single transaction.

To mitigate this limitation, several techniques have been proposed for batching multiple proofs into a single compact proof. More generally, these approaches securely combine multiple instances of a Sigma protocol, reducing the amortized proof size per eCash note. This can be achieved either through folding mechanisms, such as those employed by Bulletproofs for non-linear relations, or through proof amortization techniques designed for parallelized linear relations.

Below we can see how the batching would work. There are still open discussions concerning the security of the protocol when batching is applied both in terms of the provided proofs as well as the anonymous credential

scheme. For a single instance of the protocol we have the following where $x = (m_1, m_2, m_3, r_p, r_m, r_1, r_2, r_3)$ and $h = H_{G_1}(c_m)$.

$$\varphi_h(\vec{x}) = \vec{Y}, \quad \vec{R} = \varphi_h(\vec{\rho}), \quad c = H_s(\vec{Y} \parallel \vec{R}), \quad \vec{s} = \vec{\rho} + c\vec{x}, \quad \varphi_h(\vec{s}) \stackrel{?}{=} c\vec{Y} + \vec{R}$$

For the batching scenario for up to n outputs then the scheme changes as following:

$$\varphi_h(\vec{x}_i) = \vec{Y}_i, \quad \vec{R}_i = \varphi_h(\vec{\rho}_i) \text{ s.t. } h = H_{G_1}(c_m^1 \parallel c_m^2 \cdots \parallel c_m^n)$$

We will then derive the challenge for each instance as below:

$$c_i = H_s(\vec{Y}_1 \parallel \cdots \parallel \vec{Y}_n \parallel R \parallel i)$$

The proofs can then be constructed for the whole batch as below:

$$\vec{R} = \sum_{i=1}^n \vec{R}_i, \quad \vec{s} = \sum_{i=1}^n \vec{\rho}_i + c_i \vec{x}_i$$

The prover will then send $(\vec{R}, \vec{s})$ as the proof along with the credentials $(\vec{Y}_1, \vec{Y}_2, \dots, \vec{Y}_n)$. The verifier will then compute (h, c_i) internally and check:

$$\varphi_h(s) = \sum_{i=1}^n c_i \vec{Y}_i + \vec{R}$$

Since the value of an eCash note is hidden using cryptographic commitments, the system must additionally prove that the committed amount is valid without revealing it. In particular, range proofs are required to demonstrate that the value lies within an accepted interval (e.g., $(0 \leq v < 2^{64})$). Without such proofs, an adversary could exploit modular arithmetic to commit to negative or out-of-range values, enabling underflow (or overflow) attacks that effectively create value from nothing while still satisfying balance equations. To efficiently enforce these constraints, this work employs Bulletproofs, which provide short, non-interactive zero-knowledge range proofs without requiring a trusted setup. Bulletproofs offer logarithmic proof sizes and efficient verification, making them well suited for confidential eCash systems where both privacy and practicality are essential.

3.2 Blind Signing

Each mint upon receiving an issuance request checks the proofs provided by the client and if valid will sign the notes as below:

$$\tilde{\sigma} = x_0 h + \sum_{j=1}^3 x_j c_j = \underbrace{\left(x_0 + \sum_{j=1}^3 x_j m_j \right)}_{\text{PS signature } \sigma} h + \underbrace{\sum_{j=1}^3 r_j (x_j g_1)}_{\text{blinding term}}.$$

It's worth to mention that the signature from each mint is only partially valid and needs to then be aggregated by the client with at least $t - 1$ other valid shares to construct a valid signature/note.

3.3 Unblinding and Signature Aggregating

Each client then upon receiving a partial signature from a mint will first unblind it by removing the blinding term as below and then validate the partial signature against the mint's public keys.

$$\sigma = \tilde{\sigma} - \sum_{j=1}^3 r_j(x_j g_1)$$

The client gathers t valid partial signatures and aggregates them using Lagrange interpolation using the corresponding mints' indexes j in the set S consisting of the valid gathered partial signatures:

$$s = \left(h, \prod_{i \in S} \sigma_i^{\lambda_i} \right), \quad \lambda_i = \prod_{\substack{j \in S \\ j \neq i}} \frac{0-j}{i-j} \pmod{p}.$$

3.4 Spending

For the scheme to provide unlinkability, each client **must** reblind its signature to avoid any observer being able to connect the spending with the issuance. The re-randomization happens using two freshly chosen random values r and r' and is carried out to compute the blinded signature s' as following:

$$s' = (h', \sigma', V) = (rh, r\sigma + rr'h, rg_2)$$

This blinding not-only avoids directly linking the reblinded signature with the previously issued one but also avoids any linkability within eCash notes created within the same batch, the ones sharing their generator h , using cross-pairing attacks.

After reblinding the aggregated signature, the client can then spend its eCash by providing (s', K', P, x) constructed as below:

$$P = m_1 G_p + r'_p H_p \in \mathbb{G}_1, \quad K' = m_1 X_1 + m_2 X_2 \in \mathbb{G}_2$$

K' acts as the nullifier of the corresponding eCash, since its value is bonded to the eCash's value and serial number. Keeping track of spent K' by the mints prevents any double spending by a malicious client. Showing each eCash value at spend time also obligates the client to show a zero-knowledge proof of knowledge of m_1 and m_2 corresponding to the eCash being spent as well as proving that the Pedersen's commitment is created from the same value m_1 as the one in the eCash.

$$\psi : \mathbb{K}^3 \rightarrow \mathbb{G}_1 \times \mathbb{G}_2,$$

$$\psi(m_1, m_2, r'_p) = (m_1 G_p + r'_p H_p, m_1 X_1 + m_2 X_2) = (P, K')$$

Upon receiving the proof and the provided elements from a client, a mint has to verify that the proof is correct as well as verifying the signature on the provided eCash which is carried out as below:

$$m_3 = \text{Hash}(x) \wedge m_3 \neq 0$$

$$e(h', X_0 + K' + m_3 X_3 + V) = e(\sigma', g_2)$$

4 Benchmarks

This section evaluates the performance of the proposed confidential federated eCash protocol through a series of benchmarks covering its main operations. The analysis considers the computational cost incurred by both the prover and the verifier, the size of the generated proofs, and the storage requirements of an eCash note throughout its lifecycle. Additionally, implementation measurements are presented for each protocol phase to assess the protocol’s practicality and identify its computational bottlenecks.

4.1 Proof Sizes

The proof sizes differ significantly between the issuance and spending phases due to the different security requirements of each operation. During issuance, the user must prove the correctness of several committed values while preserving their confidentiality. When multiple eCash notes are requested simultaneously, these proofs can be batched by aggregating multiple Sigma protocol instances into a single proof. This substantially reduces the amortized proof size per note, making large issuance requests considerably more practical than verifying an independent proof for every note. For a batched instance of the issuance requests the proof size will remain constant based on the number of issuance request with a size of 496B.

In contrast, the spending phase only requires the user to demonstrate ownership of a valid eCash note. This can be achieved using the short Sigma proof of knowledge underlying the Schnorr identification protocol, which consists of a single commitment and response together with the challenge derived via the Fiat–Shamir transform. As a result, the spending proof remains compact and incurs only a small communication overhead, making it well suited for frequent payment transactions. Although the proofs for spending are more compact than the naive version the proof size still grows linearly with the number of eCash notes being spent. The following table shows the proof sizes for each of these phases with G_1 and G_2 representing group elements in the paired groups and s denotes a 32B scalar.

This values for an example of 100 eCash notes as both input and output in a transaction will take $\approx 72KB$ which is only slightly over the $64KB$ consensus of Fedimint.

Table 1: Proof size as a function of the number of eCash notes n . The range proofs are assumed to be for a 32-bit range

Phase	Proof Size	Growth
Issuance (batched zk proof)	$5G_1 + 8s$	$\mathcal{O}(1)$
Issuance (batched range proofs)	$2 \times \log_2(32 \times n)G_1 + 9G_1 + 2s$	$\mathcal{O}(\log_2(n))$
Spending	$4 \times n \times s$	$\mathcal{O}(n)$

5 Future Work

1. Add a table of notation.
2. Add security assumptions made in the scheme.
3. Add the description and design of the peg-in and peg-out operations when joining the federation.
4. Analyze the possible attack carried out by moving the attributes in the signature because of the shared base, h , in a batched issuance.
5. Add and check the bounds provided for proof batching.
6. Add the possible attacks for the shared base when batching and their mitigation.
7. Add acknowledgments.

References

- [1] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, “Co-conut: Threshold issuance selective disclosure credentials with applications to distributed ledgers,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2019.
- [2] Nym Technologies SA, “The nym network.” <https://nymtech.net/nym-whitepaper.pdf>, 2021. Accessed: 2026-07-28.